



UTPL
UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA

Análisis de Algoritmos

Ciencias de la Computación

Abril – Agosto / 2025

Unidad 1

1. Fundamentos

1.1. Conceptos de algoritmia

1.2. Análisis de algoritmos

1.3. Diseño de algoritmos

Semana 1

- **Agenda**

- ✦ Clases presenciales:

- Presentación del plan docente

- Presentación del tema

- Desarrollo de ejercicios

- ✦ Tutoría

- ✦ Preparación del repositorio de trabajo

- ✦ Lectura de los temas de la semana

Conceptos de algoritmia

¿Qué son los algoritmos?



Conceptos de algoritmia

Join at menti.com | use code 4844 0816

Mentimeter

¿Qué es un algoritmo?

18 responses



pasos a seguir
órdenes
conjunto
orden
instrucciones
estructura
proceso
secuencia
procedimiento
conjunto de pasos
pasos ordenados



Conceptos de algoritmia

¿Qué son los algoritmos?

Algoritmo es un conjunto de reglas para efectuar algún cálculo, bien sea a mano o, más frecuentemente, en una máquina

Brassard & Bratley, 2012



Se dice entonces que un algoritmo calcula una función f . La entrada es una variable independiente básica en relación a la que se producen las salidas del algoritmo, y también los análisis de tiempo y espacio (Valenzuela, 2003).

Conceptos de algoritmia

¿Qué son los algoritmos?

Por ejemplo, suponga que necesita clasificar una secuencia de números en un orden creciente

Input: Secuencia de n números $[a_1, a_2, \dots, a_n]$

Output: Una permutación (reordenamiento) $[a'_1, a'_2, \dots, a'_n]$ de la secuencia

Input: $[14, 13, 20, 16, 18]$

Output: $[13, 14, 16, 18, 20]$

Instancia



En general, una instancia de un problema consiste en la entrada (que satisface las restricciones impuestas en el enunciado del problema) necesaria para calcular una solución al problema.

Conceptos de algoritmia

- **Algoritmia:**
 - ♦ Estudio de los algoritmos
 - ♦ Ciencia que permite evaluar el efecto de factores externos sobre los algoritmos disponibles (elegir de acuerdo a circunstancias particulares)
 - ♦ Ciencia que indica la forma de diseñar un nuevo algoritmo para una tarea concreta.
- “Tratamiento sistemático de técnicas fundamentales para el diseño y análisis de algoritmos eficientes”

Conceptos de algoritmia

- ✦ Computadores cada vez más rápidos y a más bajo precio:

Se resuelven problemas de cálculo antes impensables.

Inconscientemente se tiende a restar importancia al concepto de **eficiencia**.

Existen problemas que seguirán siendo intratables si se aplican ciertos algoritmos por mucho que se aceleren los computadores, por tanto de imprescindible la importancia de nuevas soluciones **eficientes**.

Conceptos de algoritmia

Ejemplo

Multiplicación de dos enteros positivos con lápiz y papel

				9	8	1
			1	2	3	4
			3	9	2	4
		2	9	4	3	
	1	9	6	2		
	9	8	1			
1	2	1	0	5	5	4

En Ecuador

				9	8	1
			1	2	3	4
	9	8	1			
	1	9	6	2		
		2	9	4	3	
			3	9	2	4
1	2	1	0	5	5	4

En Inglaterra

Algoritmo de multiplicación “a la rusa”

✓	981	1234	1234
	490	2468	
✓	245	4936	4936
	122	9872	
✓	61	19744	19744
	30	39488	
✓	15	78976	78976
✓	7	157952	157952
✓	3	315904	315904
✓	1	631808	631808
			<u>1210554</u>

Ventaja: no hay que almacenar los productos parciales.
Sólo hay que saber sumar y dividir por 2.

Conceptos de algoritmia

Otro algoritmo

Ambos números tengan igual numero de cifras y que éste sea potencia de 2. (981 x 1234)

Luego partimos ambos números por la mitad y hacemos cuatro productos:

	multiplicar		desplazar	resultado
1)	09	12	4	108
2)	09	34	2	306 . .
3)	81	12	2	972 . .
4)	81	34	0	2754
				1210554

dobles del n° de cifras

n° de cifras

Hemos reducido un producto de números de 4 cifras en cuatro productos de números de 2 cifras, varios desplazamientos y una suma.

Conceptos de algoritmia

- Ideas:

- ✦ Incluso para un problema tan básicos pueden construirse MUCHAS soluciones.
- ✦ El método clásico lo usamos con lápiz y papel porque nos resulta muy familiar (lo que se aprende en la infancia...).
- ✦ El método “a la rusa” se implementa en hardware en los computadores por la naturaleza elemental de los cálculos intermedios.
- ✦ El método de divide y vencerás es más rápido si se quiere multiplicar números grandes.

Conceptos de algoritmia

- En conclusión
 - ✦ La **algoritmia** estudia las propiedades de los algoritmos y nos ayuda a **elegir** la solución más adecuada en cada situación.
 - ✦ En muchos casos, una buena elección **ahorra tiempo y dinero**.
 - ✦ En algunos casos, una buena elección marca la diferencia entre **poder** resolver un problema y **no poder** hacerlo.

Eficiencia

- Un algoritmo debe ser rápido y usar la menor cantidad de recursos. Es una relación entre los recursos consumidos, fundamentalmente **tiempo** y **memoria** versus los productos obtenidos.

Eficiencia de Algoritmos

- Pregunta:
 - ✦ ¿Cómo decidir entre varios algoritmos cuál es el adecuado?
- Análisis de algoritmos:
 - ✦ Enfoque empírico
 - ✦ Enfoque teórico

Análisis empírico de algoritmos

- Escribe el programa
- Incluye instrucciones para medir el tiempo de ejecución
- Ejecuta el programa con entradas de diferentes tamaños
- Representa gráficamente los resultados

Análisis empírico de algoritmos

- Ejemplo:

Dado un número n , desarrolle un método para sumar de 1 a n

1. Escribe el programa:

```
public static long Suma(long n) {  
    long Resultado = 0;  
    for (long i=1; i<n; i++)  
        Resultado += i;  
    return Resultado;  
}
```

Análisis empírico de algoritmos

2. Incluye instrucciones para medir el tiempo de ejecución

```
public static long Suma(long n) {  
    long Inicio = System.currentTimeMillis();  
    long Resultado = 0;  
    for (long i=1; i<n; i++)  
        Resultado += i;  
    long Final = System.currentTimeMillis();  
    long Total = Final - Inicio;  
    System.out.println("Tiempo: " + Total + " ns");  
    return Resultado;  
}
```

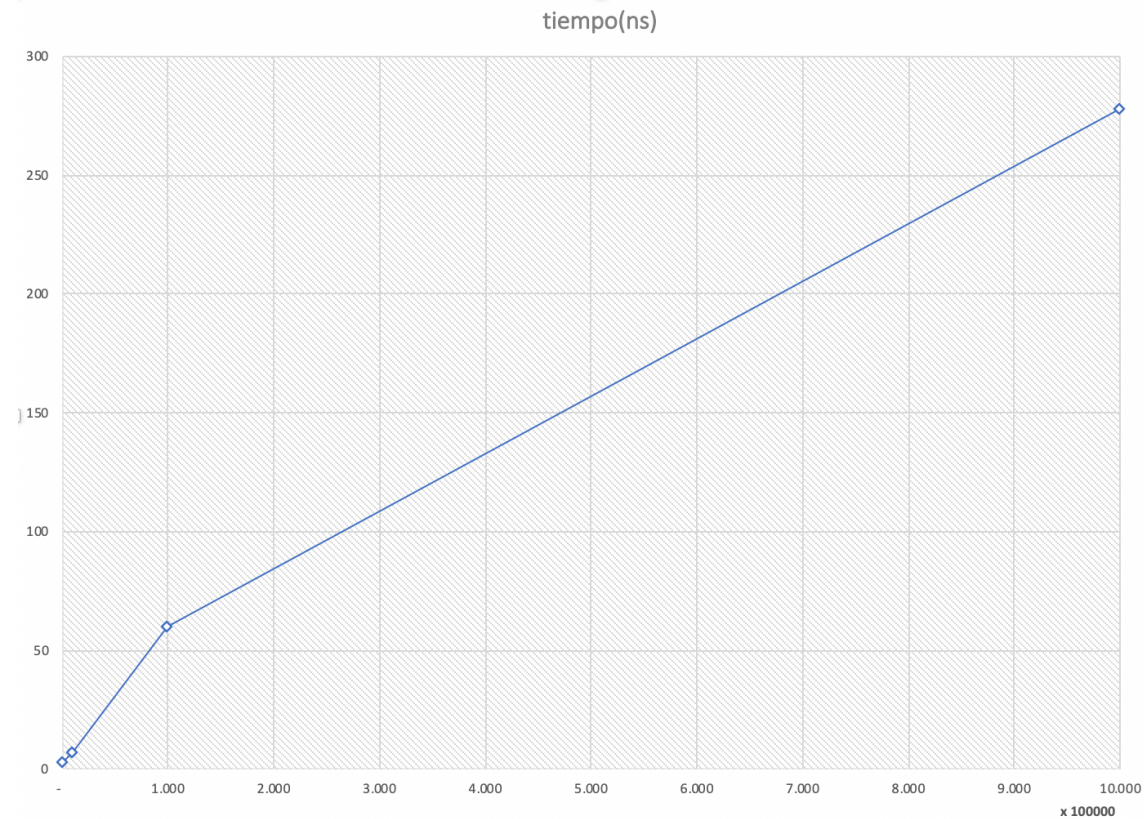
Análisis empírico de algoritmos

3. Ejecuta el programa con entradas de diferente tamaño

n	tiempo(ns)
1.000.000	3
10.000.000	7
100.000.000	60
1.000.000.000	278

Análisis empírico de algoritmos

4. Representa gráficamente los resultados



Análisis empírico de algoritmos

- ¿Otros algoritmos que resuelvan este problema?

```
public static long Suma(long n) {  
    long Resultado = 0;  
    for (long i=1; i<n; i++)  
        Resultado += i;  
    return Resultado;  
}
```

- ✦ La solución de Gauss para sumar números del 1 al n .

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

Si n es par, hay $n/2$ pares
Si n es impar, hay $(n-1)/2$ pares
Cada par suma $n+1$

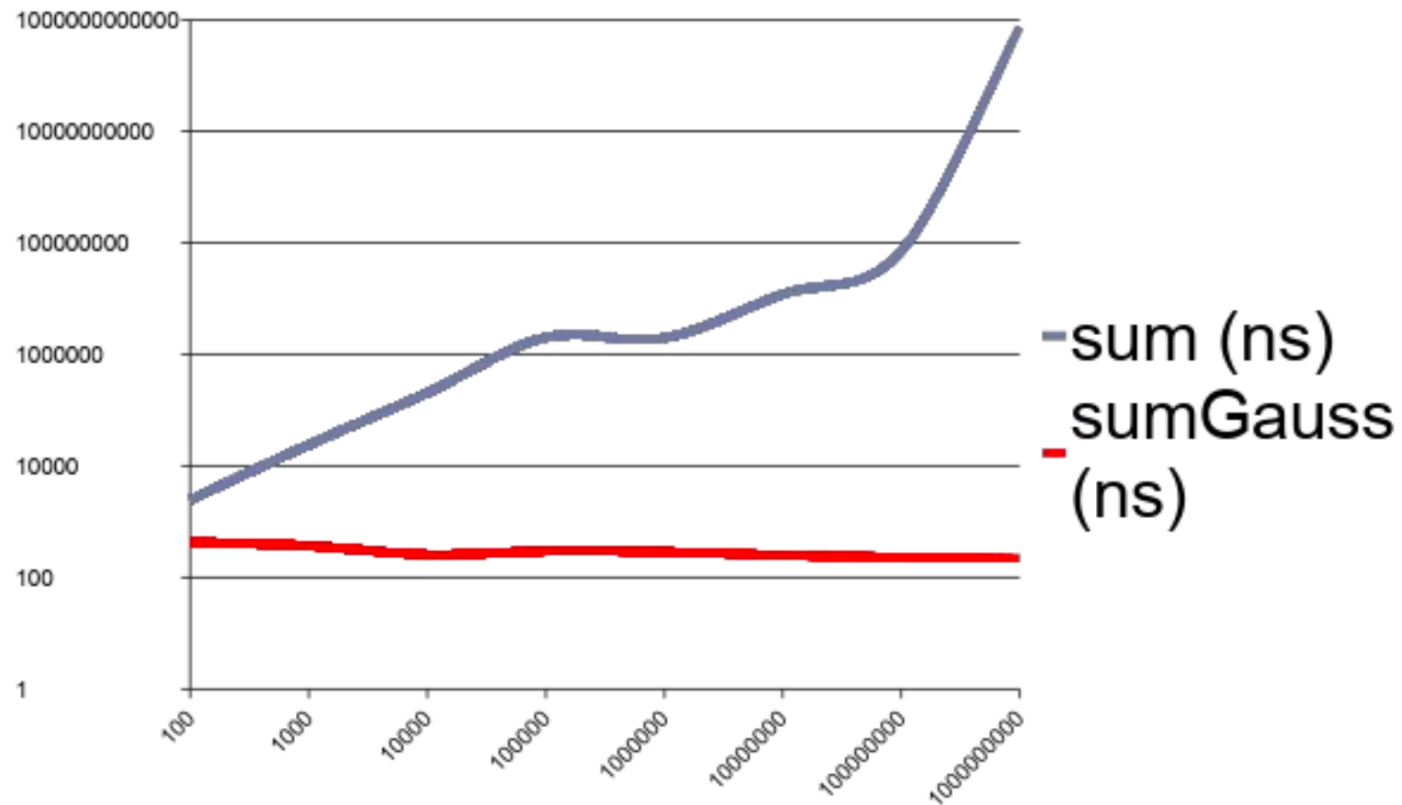
Análisis empírico de algoritmos

- Ejemplo

```
public static long Suma(long n) {  
    long Inicio = System.currentTimeMillis();  
    long Resultado = n*(n+1)/2;  
    long Final = System.currentTimeMillis();  
    Total = Final - Inicio;  
    System.out.println("Tiempo: " + Total + " ns");  
    return Resultado;  
}
```

n	tiempo (ns)
100	436
1.000	371
10.000	259
100.000	298
1.000.000	290
10.000.000	250
100.000.000	
0	233
1.000.000.000	
00	222

Análisis empírico de algoritmos



Análisis empírico de algoritmos

- Desventajas:
 - ✦ Es necesario implementar el algoritmo
 - ✦ Los resultados pueden no ser indicativos para otras entradas
 - ✦ Se requiere del mismo entorno para comparar dos algoritmos

Análisis teórico de algoritmos

- Determinar matemáticamente la cantidad de recursos necesarios para cada uno de los algoritmos en función del tamaño de los casos considerados.
- Función de “Tiempo de ejecución”
- Toma en cuenta todas las entradas posibles
- Pseudocódigo
 - ✦ Define $T(n)$ en función del tiempo de ejecución.
 - ✦ Independencia con el entorno hardware/software

Análisis teórico de algoritmos

- Ejecución del tiempo de ejecución:
 - ✦ Representar el tiempo de ejecución de un algoritmo en función del tamaño de la entrada.
 - ✦ $T(n)$ = número de operaciones ejecutadas por un algoritmo para procesar una entrada según el tamaño
 - ✦ Las operaciones primitivas toman una cantidad constante de tiempo
 - ✦ Reglas de estimaciones:
Secuencial, bucles, bucles anidados, If/else,

Análisis teórico de algoritmos

- Principio de invariancia: “Dos implementaciones distintas de un mismo algoritmo no diferirán en su eficiencia en más de alguna constante multiplicativa”.
 - ✦ Sea: $t_1(n)$ y $t_2(n)$ segundos
 - ✦ Entonces existen constantes positivas c y d , tales que:
 - ✦ $t_1(n) \leq c t_2(n)$ y $t_2(n) \leq d t_1(n)$ siempre que n sea suficientemente grande.

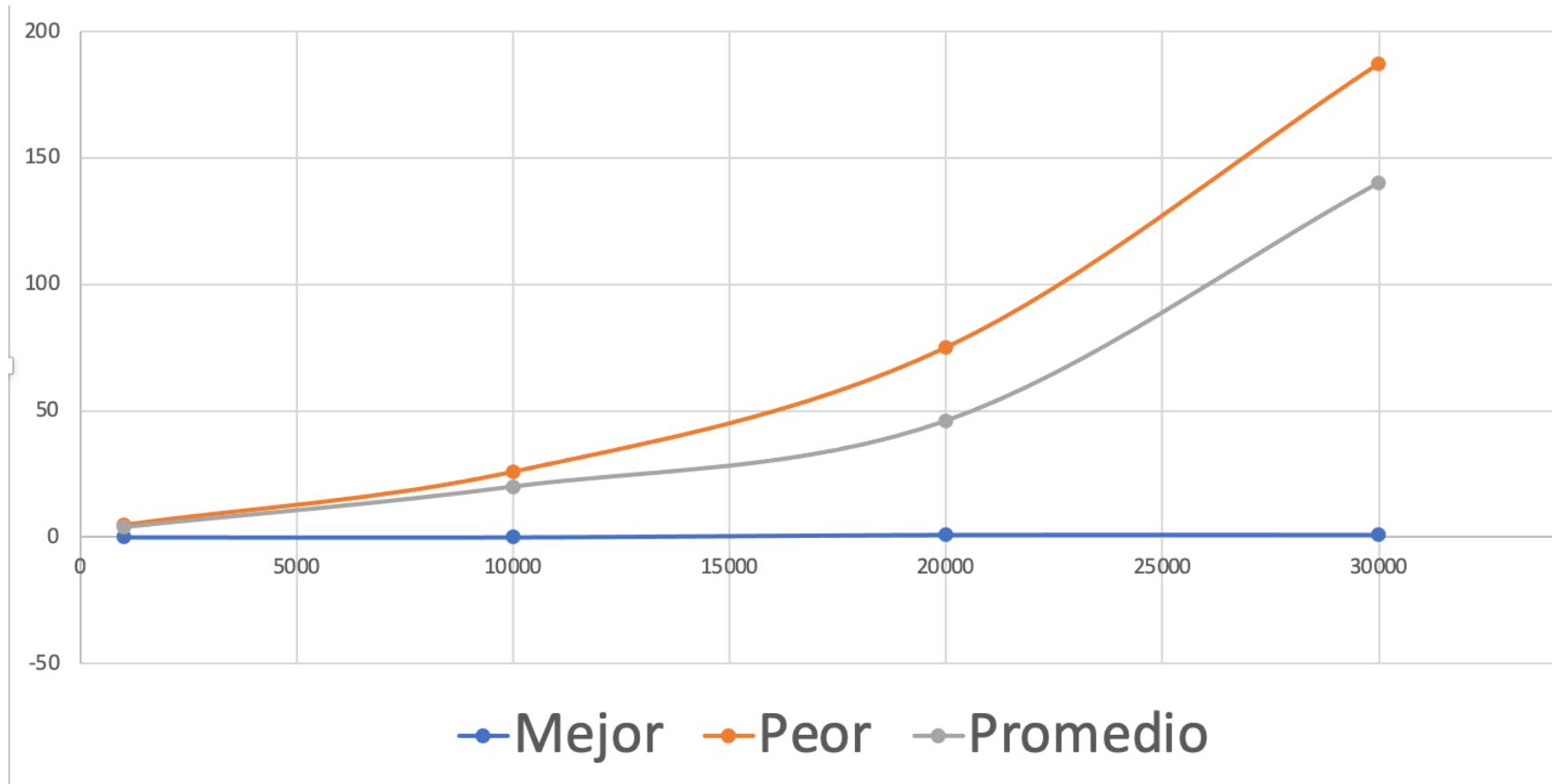
Caso medio y Caso peor

- Tiempo y espacio pueden variar considerablemente entre dos ejemplares distintos del mismo tamaño.
 - ✦ Ejemplo: ordenación por inserción y selección (texto base)
 - ✦ Taller: Implementar los algoritmos antes indicados

Análisis

- Tipos de análisis:
 - ✦ **Peor caso:** indica el mayor tiempo obtenido, teniendo en consideración todas las entradas posibles.
 - ✦ **Mejor caso:** indica el menor tiempo obtenido, teniendo en consideración todas las entradas posibles.
 - ✦ **Media:** (Caso promedio), indica el tiempo medio obtenido, considerando todas las entradas posibles

Análisis



Análisis

Ejemplo

Sea A una lista de n elementos $A_1, A_2, A_3, \dots, A_n$. Ordenar significa permutar estos elementos de tal forma que los mismos queden de acuerdo con un orden preestablecido.

Ascendente $A_1 \leq A_2 \leq A_3 \dots \leq A_n$

Descendente $A_1 \geq A_2 \geq \dots \geq A_n$

Caso peor: Que el vector esté ordenado en sentido inverso.

Caso mejor: Que el vector esté ordenado.

Caso medio: Cuando el vector esté desordenado aleatoriamente.

Ejercicio

```

procedimiento insertar ( $T[1...n]$ )
  para  $i \leftarrow 2$  hasta  $n$  hacer
     $x \leftarrow T[i]; j \leftarrow i - 1$ 
    mientras  $j > 0$  y  $x < T[j]$  hacer  $T[j+1] \leftarrow T[j]$ 
     $j \leftarrow j - 1$ 
     $T[j+1] \leftarrow x$ 

```

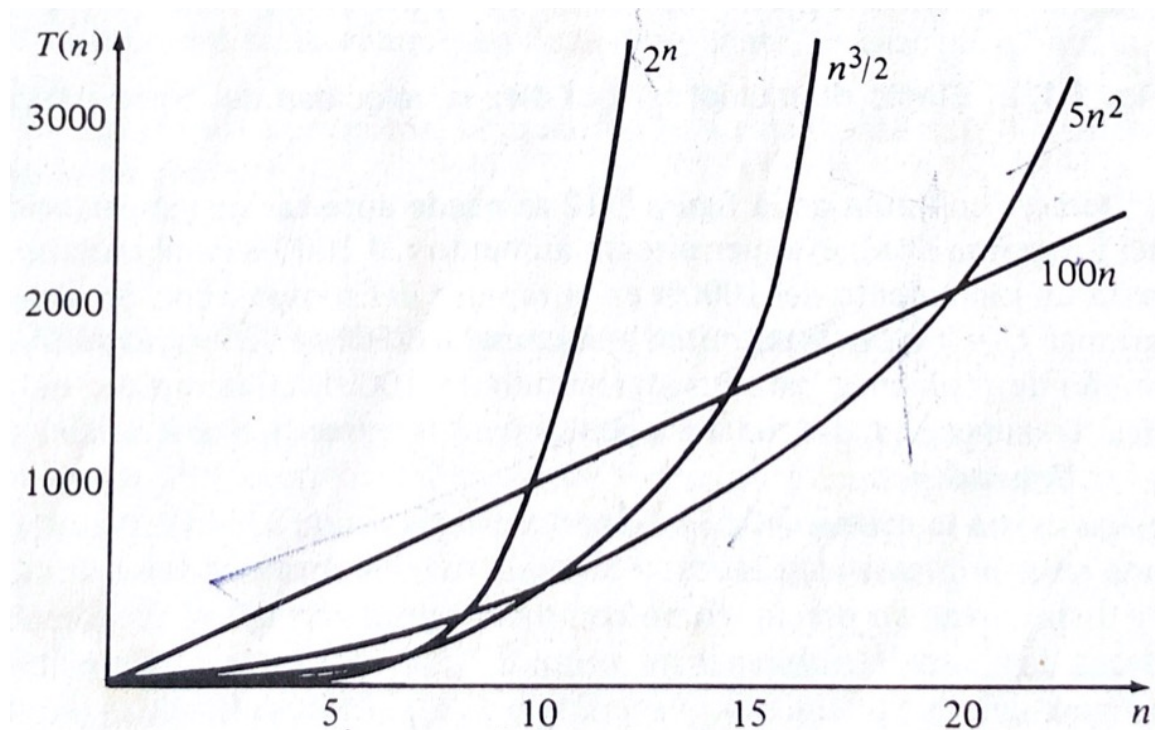
```

procedimiento seleccionar ( $T[1 \dots n]$ )
  para  $i \leftarrow 1$  hasta  $n-1$  hacer
     $minj \leftarrow i$ ;  $minx \leftarrow T[i]$ 
    para  $j \leftarrow i+1$  hasta  $n$  hacer
      si  $T[j] < minx$  entonces  $minj \leftarrow j$ 
       $minx \leftarrow T[j]$ 
     $T[minj] \leftarrow T[i]$ 
     $T[i] \leftarrow minx$ 

```


Caso medio y Caso peor

- Ejemplo: Tiempos de ejecución de cuatro algoritmos



Caso medio y Caso peor

Efecto de multiplicar por 10 la velocidad del computador

t(n)	Unidades de tiempo		Diferencia
	1000	10000	
$100n$			
$5n^2$			
$\frac{n^3}{2}$			
2^n			

Notación

- Teoría de conjuntos:

- ✦ Cardinalidad

- ✦ $\mathbb{N} = \{0, 1, 2, 3, \dots\}$

$$X \setminus Y = \{z \mid z \in X \text{ pero } z \notin Y\}$$

Notación

- Notación matemática

- ✦ Símbolos que se utilizan: $[i..j], \forall, \exists, lg, [x], R^{\geq 0}$

- ✦ Cálculo proposicional

Valores de verdad (verdadero o falso)

Variable booleana

Conjunción, disyunción, negación

Construir fórmulas booleanas a partir de variables booleanas, constantes, conectivas y paréntesis

Notación

- Enteros, Reales e intervalos
 - ♦ $Z = \{\dots, -2, -1, 0, 1, 2, \dots\}$, N
 $R^+ = \{x \in R | x > 0\}$
- Funciones y relaciones
- Cuantificadores
- Sumas y productos
- Logaritmos

Demostración

- Para decidir el algoritmos más adecuado, es necesario:
 - ✦ Establecer las propiedad matemáticas de los algoritmos
 - ✦ Demostrar matemáticamente
 - ✦ Dos técnicas útiles en Algoritmia:
 - Demostración por contradicción
 - Demostración por inducción matemática

Demostración por contradicción

- Prueba indirecta.
- Demostrar la veracidad de una sentencia demostrando que su negación da lugar a una contradicción.
- Ejemplo:

Teorema 1.5.1 (Euclides) *Existen infinitos números primos*

Demostración por contradicción

Demostración. Sea P el conjunto de los números primos. Supongamos para buscar una contradicción que P es un conjunto finito. El conjunto P no es vacío, porque contiene al menos el entero 2. Dado que P es finito y no está vacío, tiene sentido multiplicar todos sus elementos. Sea x ese producto, y sea y el valor $x + 1$. Consideremos el menor entero d que es mayor que 1 y que es el divisor de y . Este entero existe ciertamente por cuanto y es mayor que 1 y no exigimos que d sea distinto de y . Obsérvese en primer lugar que d en sí es primo, porque en caso contrario todo divisor propio de d dividiría también a y , y sería más pequeño que d , que estaría en contradicción con la definición de d . (¿Ha observado el lector que la sentencia anterior es, en sí misma, una demostración por contradicción, anidada en el esquema general?) Por tanto, de acuerdo con nuestra suposición de que P contiene absolutamente todos los números primos, d pertenece a P . Esto demuestra que d es también divisor de x , puesto que x es el producto de una colección de enteros que contiene a d . Hemos llegado a la conclusión de que d divide exactamente tanto a x como a y . Pero se recordará que $y = x + 1$. Por tanto, hemos obtenido un entero d más grande que 1 y que divide a dos enteros consecutivos x e y . Esto es claramente imposible: si realmente d divide a x , entonces la división de y por d dejará necesariamente 1 como resto. La conclusión ineludible es que la suposición original era igualmente imposible. Pero la suposición original era que el conjunto P de todos los primos es finito, y por tanto su imposibilidad indica que el conjunto P es, de hecho, infinito.

Ejemplo

función *Nuevoprime* (P : conjunto de enteros)
{El argumento P debería ser un conjunto de primos no vacío finito}
 $x \leftarrow$ producto de los elementos de P
 $y \leftarrow x + 1$
 $d \leftarrow 1$
 repetir $d \leftarrow d+1$ hasta que d divide a y
 devolver d

Ejemplo

Valor: 211

```
public static int NuevoPrimo(int p[]) {  
    int x = 1;  
    for (int i=0; i<p.length; i++)  
        x = x * p[i];  
    int y = x + 1;  
    int d = 1;  
    do {  
        d++;  
    } while (y%d != 0);  
    return d;  
}  
  
public static void main(String[] args) {  
    int p[] = {2, 3, 5, 7};  
    System.out.println("Valor: " + NuevoPrimo(p));  
}
```

Demostración por Inducción matemática

- Demostrar propiedades acerca de corrección y eficiencia de algoritmos.
- Determinar qué propiedades es preciso probar.
- Ejemplo:

Dado el polinomio: $p(n) = n^2 + n + 41$

Si computamos: $p(0), p(1), p(2), \dots, p(10)$

Obtenemos: 41, 43, 47, 53, 61, 71, 83, 97, 113, 131, 151

Inferir por Inducción matemática que:

$p(n)$ es primo para todos los valores enteros de n

Pero: $p(40) = 1681 = 41^2 \rightarrow$ es compuesto.

Principio de Inducción matemática

- Es una técnica de demostración lógica muy utilizada en matemáticas, especialmente para probar afirmaciones que se aplican a todos los números naturales (o a una parte de ellos, a partir de cierto punto).
- Tres pasos:
 - ✦ Caso base
 - ✦ Hipótesis de inducción
 - ✦ Demostración

Ejemplo

Problema. Demuestre que la suma de los primeros n números naturales es $\frac{n(n+1)}{2}$.

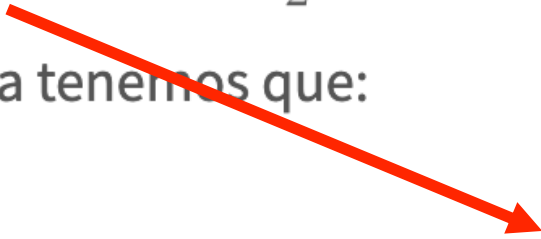
Solución. Vemos que la proposición que tenemos que demostrar es $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.

Paso 1: Para $n = 1$ tenemos que $\sum_{i=1}^1 i = 1$, y por otro lado $\frac{1(1+1)}{2} = 1$, por lo tanto la proposición es cierta para $n = 1$.

Paso 2: Supongamos que la proposición es verdadera para cualquier $k \in \mathbb{N}$, es decir, supongamos que $\sum_{i=1}^k i = \frac{k(k+1)}{2}$.

Ejemplo

- **Paso 3:** Probemos que la proposición es cierta para $k + 1$, es decir, probemos que $\sum_{i=1}^{k+1} i = \frac{(k+1)((k+1)+1)}{2} = \frac{(k+1)(k+2)}{2}$. ¹ Al separar el último término de la suma tenemos que:


$$\sum_{i=1}^{k+1} i = \sum_{i=1}^k i + (k+1)$$

Pero por la hipótesis de inducción tenemos ahora:

$$\sum_{i=1}^{k+1} i = \frac{k(k+1)}{2} + (k+1)$$

Ejemplo

Lo que resta es simple álgebra para llegar a la expresión deseada, factorizando y reacomodando tenemos que:

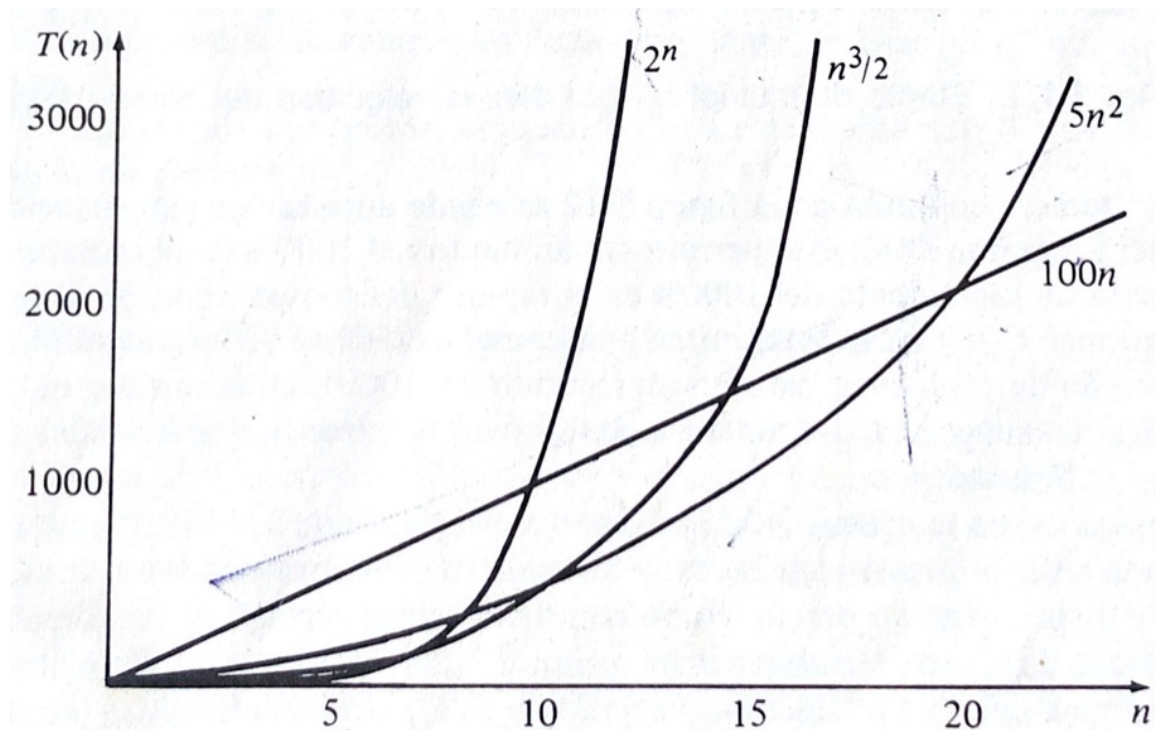
$$\sum_{i=1}^{k+1} i = (k+1) \left(\frac{k}{2} + 1 \right) = \frac{(k+1)(k+2)}{2} \quad 1$$

Finalmente, por el principio de inducción matemática, se cumple que

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}, \text{ para toda } n \in \mathbb{N}. \square$$

Análisis de algoritmos

- Ejemplo: Tiempos de ejecución de cuatro algoritmos



Eficiencia de Algoritmos

- Qué tan bien resuelve un algoritmo un problema en términos de tiempo y espacio (memoria).
- Un algoritmo eficiente resuelve el problema más rápido, lo cual es crucial cuando:
 - ✦ El tamaño de los datos es muy grande.
 - ✦ Se requiere una respuesta en tiempo real (por ejemplo, en videojuegos, transacciones bancarias, navegación GPS).
 - ✦ El sistema tiene recursos limitados o muchos usuarios simultáneos.

Eficiencia de Algoritmos

- Ejemplo:
 - ✦ Ordenar un millón de elementos con un algoritmo de eficiencia:

$$O(n^2)$$

$$O(n \log n)$$

- ✦ Puede hacer una diferencia de **segundos frente a horas**.

Taller

```
MERGE( $A, p, q, r$ )
1   $n_L = q - p + 1$       // length of  $A[p : q]$ 
2   $n_R = r - q$           // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$  // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$  // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                 //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                 //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                 //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //   copy the smallest unmerged element back into  $A[p : r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19          $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //   remainder of the other to the end of  $A[p : r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 
```



UTPL
UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA